ASC Student Supercomputer Challenge 2026 Preliminary Contest Proposal

Fuzhou University
Team: FZUASC Team Advisor: Xiangwen Liao Date: 2026.1.20
Members: Hao Yuan, Yali Pu, Jinchao Wu, Zhiyuan Chen, Xun Zhang

***Abstract*—In the preliminary contest, we utilized the Fujian Supercomputing Center cluster. Our system design centered on an AMD EPYC 9K84 cluster to provide tailored solutions for the Embodied World Model Optimization and the Numerical Relativity Code AMSS-NCKU Optimization challenges. System performance was evaluated via HPL and HPCG benchmarks. For the World Model task, we implemented combined strategies to accelerate the inference process while satisfying all constraints, achieving a speedup of $7.58\times$. For the AMSS-NCKU challenge, we mitigated the inherent load imbalance, optimized memory access and optimized communication, ultimately reaching a speedup of $4.33\times$.**

***Index Terms*—HPC, World Model, Numerical Relativity**

I. Supercomputing-related hardware and software platforms

The team is affiliated with the Fujian Supercomputing Center at Fuzhou University, which serves as one of the key initiatives under the construction of the "Digital Fujian" public infrastructure service platform.

The project was officially approved by the Fujian Provincial Development and Reform Commission in January 2008, with a total planned investment of RMB 16.41 million. The high-performance computing (HPC) public service platform is characterized by comprehensive functionality, high operational efficiency, and a well-established system architecture. Led by Fuzhou University, the platform also integrates the computational resources of the already established High-Performance Computing Center at Xiamen University.

Following a period of trial operation, Phase II of the Fujian Supercomputing Center has been formally put into service. Benefiting from these developments, one of the Center's distinguishing features is its strong high-performance parallel computing capability. The system consists of 211 computing nodes, achieving a peak performance of up to 800 teraflops (800 trillion floating-point operations per second).

The second phase of the project is designed to provide computational services for sectors including meteorology, environmental protection, marine science, healthcare, and automotive engineering. It also supports technical collaboration among government departments, enterprises, public institutions, universities, and research organizations within Fujian Province. The Fujian Supercomputing Center aims to develop into a nationally leading public resource service center for high-performance computing, a technological support center for HPC innovation, and a value-added service center for advanced computational applications.

TABLE I
HARDWARE RESOURCES

Name	Details
System peak	13.58 Tflops
Computing nodes	290 4-way Intel Xeon Processors
Storage node	12 4-way Intel Xeon Processors
Acess node	4 4-way Intel Xeon Processors
CPU	Intel Xeon E5440, 2.83GHz × 124
Total system memory	2448 GB
Total disk capacity	20 TB
Architecture	Fluster, InfiBand
Operating System	CentOS 5.2

TABLE II
SOFTWARE RESOURCES

Name	Details
Languages	C, C++, Fortran77/90/95, Java
Compilers	GCC, Javac, tcl/tk, Perl
Math Libraries	GOTOBlast, OpenBLAS, Intel MKL
Parallel Environments	OpenMPI, Ofed, Ray
Tools Softwares	OpenLDAP2.3, Python3.12
System Management	Sun Grid Engine 6.1

A. Hardware resources and software resources

The hardware and software resources of the Fujian Supercomputing Center at Fuzhou University are presented in Table I and Table II, respectively.

B. Supercomputing-related activities

The university has established a relatively comprehensive education and training system centered on high-performance computing (HPC), aimed at enhancing the supercomputing utilization capabilities of both students and researchers through formal coursework, specialized training, and student organizations.

The core courses include Introduction to High-Performance Computing, High-Performance Computing Programming, and Parallel Computing, which systematically cover the funda-

mental principles of parallel computing as well as practical programming techniques.

In addition, the university regularly organizes platform training sessions and technical exchange activities. It has also founded the "Fuzhou University Supercomputing Association," which currently has more than 200 members. The association provides students with opportunities for hands-on supercomputing practice, project-based training, and technical knowledge sharing.

C. Supercomputing-related research and applications

Leveraging the supercomputing platform, the university has carried out extensive applications across multiple scientific and engineering disciplines, providing critical support for complex simulations and large-scale computational tasks.

Supercomputing resources have been widely applied in areas including advanced materials and new energy, automotive and ship manufacturing, civil engineering, meteorology and environmental science, biomedical sciences, physical chemistry, and aerospace engineering. These resources are utilized for tasks such as structural simulation, fluid dynamics analysis, quantum chemical calculations, weather forecasting, and drug research, significantly enhancing research efficiency and overall research quality.

D. Key achievements on supercomputing research

With the support of the supercomputing platform, the university has achieved significant accomplishments in terms of research contributions and scholarly outputs.

In 2014, Fuzhou University was awarded the title of "Outstanding Contributing Institution" by the China Education and Research Computer Network (CERNET) in recognition of its exceptional performance in research networking and high-performance computing applications.

In 2015, research conducted on the computational platform of the Fujian Supercomputing Center was published in the SCI-indexed journal Fluid Phase Equilibria, demonstrating the pivotal role of the supercomputing platform in promoting high-level scientific research.

II. Team Introduction

A. The building process of FZUASC Team

FZUASC was established in October 2017 and is composed of students from diverse academic backgrounds and different academic years.

The team ranked 30th in ASC17. In ASC18, FZUASC advanced to the finals for the first time and was awarded First Prize. The team subsequently won First Prize multiple times in ASC19, ASC20–21, ASC22–23, ASC24, and ASC25.

Under the guidance of our advisors, we aspire to achieve even greater success in future competitions. Guided by the principles of passion, innovation, and diligence, the FZUASC team is committed to embracing challenges and striving for continuous self-improvement. United by a shared vision, we will continue to move forward in pursuit of our goals.

B. Team members

Advisor: Xiangwen Liao

Xiangwen Liao born in 1980, Professor, CCF senior member. He received his PhD degree in computer architecture from the Institute of Computer Sciences, Chinese Academy of Sciences. His research interests include information retrieval, social media process, natural language processing, and high-performance computing.

Captain: Hao Yuan

Self introduction: A junior student from the School of Computer Science and Big Data, with a keen interest in computer applications, is mainly responsible for the AMSS-NCKU part of ASC26.

The blessing: Harnessing the ultimate power of computing to solve the world's greatest challenges.



Member: Yali Pu

Self introduction: A student at the School of Computer Science and Big Data. By debugging, experimenting, and observing system behavior, I've developed a strong interest in computer systems' underlying mechanisms, fueling my passion for deeper exploration.

The blessing: May we always keep our curiosity and spirit of exploration alive, and cherish every day.



Member: Jinchao Wu

Self introduction: A Computer Science and Big Data student dedicated to the architecture of AI. Focused on optimizing the structural foundations that power intelligent systems and solving complex challenges through systemic innovation.

The blessing: May we break through every bottleneck and continue to push the frontiers of technology together.



Member: Zhiyuan Chen

Self introduction: A Computer Science and Software Engineering student passionate about AI Infra and MLSys. Dedicated to building and optimizing the core infrastructure that empowers intelligent systems, aiming to bridge innovative algorithms with scalable, efficient real-world implementations.

The blessing: May our work in translate into solutions for real-world engineering challenges, turning complex theories into tangible impact.



Member: Xun Zhang

A junior at the School of Computer Science and Big Data, passionate about system architecture and data analytics. He enjoys exploring the mechanics of computer systems to elegantly solve complex problems, aiming to empower scientific research through advanced computing.

The blessing:May our team stay inspired to innovatively break through every bottleneck, while this competition continues to empower young talents to push the boundaries of technology.



C. Tearm slogan and logo

Our slogan is “Everything Can Be Computed, Infinite Possibilities Await” As for our logo, as you can see in the picture, our logo consists of a design and our team name. The upper part consists of three letters: “A”, “S” and “C”. Also, it shows the deep connotation of our team through the red color and the design is rich in the times, besides our team's strong dedication to ASC.

FZU ASC

Fig. 1. FZU ASC Team's Logo



Fig. 2. FZU ASC Team's Photo

III. HPC System Design

A. Overview

For the specific application of this preliminary contest, we have completed the test based on the heterogeneous cluster with GPUs and CPU-only cluster, so we provide two types of design for HPC systems as below.

B. CPU node

The configuration of each CPU node is shown in **Table III**.

C. Accelerators scheme

We propose a solution for GPUs. We learn from Preliminary Round Notification that each Inspur NF5280M6 server supports four GPUs at most. We choose the NVIDIA H100 with an estimated power consumption of 350W each. Based on this, we decided to install four H100 GPUs for 1 node so that we can limit the power consumption. We use CPU-GPU heterogeneous acceleration calculations in these nodes. The configuration is shown in .

D. System evaluation

Our proposed HPC system architecture is strategically designed to meet the rigorous computational and power constraints of the ASC competition. The system presents the following key advantages:

1) The dual Intel Xeon Platinum 8562Y+ processors offer robust support for AVX-512 instructions. This directly synergizes with our vectorized optimizations in the AMSS-NCKU application, allowing for maximum

TABLE III
CPU NODE CONFIGURATION

Item	Name	Configuration
Server	Inspur NF5290M6	CPU: **Intel Xeon Platium 8562Y+** ×2 2.80GHz, 64 cores total
		Memory: **64GB DDR5 5600MT/s** × 16
		Hard Disk: **7TB SSD SATA** × 1
		Power consumption estimation: CPU TDP 350W, Memory 7.5W, HD 10W
HCA-Card	EDR	**InfiniBand Mellanox ConnectX®-5 HCA card, single port, QSFP, EDR IB**
		Power consumption estimation: 9W
Switch	GbE switch	**10/100/1000Mb/s, 24 ports Ethernet switch**
		Power consumption estimation: 30W
	EDR-IB switch	**Switch-IB™ EDR InfiniBand, 36 QSFP port**
		Power consumption estimation: 130W
Cabel	Gigabit-CAT6	CAT copper cable, 3m
	InfiniBand Cable	InfiniBand EDR copper cable, QSFP port, cooperating with the InfiniBand switch for use

TABLE IV
GPU NODE CONFIGURATION

Item	Name	Configuration
Server	Inspur NF5290M6	CPU: **Intel Xeon Platium 8562Y+** ×2 2.80GHz, 64 cores total
		Memory: **64GB DDR5 5600MT/s** × 16
		Hard Disk: **7TB SSD SATA** × 1
		Power consumption estimation: CPU TDP 350W, Memory 7.5W, HD 10W
HCA Card	EDR	**InfiniBand Mellanox ConnectX®-5 HCA card, single port, QSFP, EDR IB**
		Power consumption estimation: 9W
Switch	GbE switch	**10/100/1000Mb/s, 24 ports Ethernet switch**
		Power consumption estimation: 30W
	EDR-IB switch	**Switch-IB™ EDR InfiniBand, 36 QSFP port**
		Power consumption estimation: 130W
Acceler-ator	GPU	**NVIDIA H100 80GB PCIE** × 4
		Power consumption per card: 350W

CPU throughput. Concurrently, the NVIDIA A800 GPUs provide the massive memory bandwidth and tensor computational power required to accelerate the UnifoLM-WMA-0 Embodied World Model inference.

2) The integration of an EDR InfiniBand network architecture guarantees high-bandwidth and low-latency inter-node communication. This is critical for minimizing MPI synchronization overhead, thereby ensuring load balancing and high parallel efficiency across distributed tasks like HPCG and AMSS-NCKU.

IV. HPL & HPCG

A. Overview

In this section, following the competition requirements, we employed HPL (High-Performance Linpack) and HPCG (High Performance Conjugate Gradients) to benchmark our cluster. Our objective was to extract the maximum system performance and investigate the trade-offs between performance and power dissipation.

We present our contributions by elaborating on the software stack, testing protocols, performance tuning strategies, and final performance evaluations.

B. Introduction of HPL & HPCG

HPL (High-Performance Linpack) [1] is the standard benchmark for evaluating the peak performance of high-performance computing systems. By solving large-scale dense systems of linear equations, it measures the system's maximum double-precision floating-point (FP64) compute capability and serves as the primary metric for the TOP500 list. It is fundamentally compute-bound, reflecting the theoretical peak performance of the underlying hardware.

HPCG (High Performance Conjugate Gradients) [2] is designed to simulate real-world scientific computing workloads. By employing a multigrid preconditioned conjugate gradient algorithm to solve sparse linear systems, it heavily exercises memory bandwidth, cache efficiency, and interconnect communication. Compared to HPL, HPCG more accurately approximates real application behavior, providing a comprehensive reflection of the system's practical capabilities in scientific computing scenarios.

C. Hardware and software environment

TABLE V
THE CONFIGURATION OF CLUSTER

Name	Details
CPU	2 × AMD EPYC 9K84 @ 2.6 GHz
GPU	8 × NVIDIA A100-SXM4-40GB

TABLE VI
SOFTWARE ENVIRONMENT

Classification	Description	Version
Operating system	Ubuntu	25.04
Compiler	Intel Compilers	2025.0
MKL	Intel MKL	2025.0
MPI	Open MPI	5.0.7

D. HPL Performance Tuning Method

We use the following methods to optimize performance of our cluster.

- use the optimal library;

- turn on compiler optimization like `-O3`;
- optimize the parameters in `HPL.dat` and some other files.

We will mainly focus on the third method, which requires tuning parameters such as `NB`, `P`, `Q`, and `N` separately.

a) *Find the base `NB` size:*

The block size `NB` determines the granularity of the computation. The value of NB $\times$ 8 must be a multiple of the cache line size, and `NB` is typically less than 256. Empirical testing indicates that the optimal choice of `NB` also depends on various factors, including communication overhead, matrix size, network interconnect, and processor speed. Typically, a few near-optimal `NB` values can be identified during single-node or single-CPU benchmarking. However, as the system scale and problem size increase, the performance of certain `NB` values may degrade. Therefore, it is best practice to select approximately three promising `NB` candidates from small-scale runs and validate them in large-scale tests.

b) *Tuning the 2D Processor Grid:*

In HPL, P and Q denote the row and column dimensions of the 2D process grid, which is used to map the block-partitioned matrix across the cluster's processors for parallel execution. The product $P \times Q$ must equal the total number of MPI processes. To minimize column-wise communication overhead, a smaller P is generally preferred. Consequently, it is common practice to configure $P \leq Q$, ideally setting P as a power of 2 to achieve optimal performance.

c) *Find largest matrix size `N`:*

In HPL, N denotes the order of the coefficient matrix to be solved, defining a global problem size of $N \times N$. It directly governs the computational scale and the total volume of floating-point operations. Generally, a larger matrix yields higher floating-point efficiency. However, it concurrently increases the memory footprint; exceeding the available physical memory will severely degrade performance. As a rule of thumb, N is typically calibrated so that the matrix occupies approximately 80% of the total system memory. Furthermore, to optimally balance computational efficiency and communication load, N should ideally be evenly divisible by NB $\times$ LCM(P, Q).

E. HPL on CPU

In our HPL benchmarking, we first identified the CPU cache line size to be 64 bytes by executing:

```
cat /proc/cpuinfo
```

Guided by this hardware specification, we configured a sequence of cache-aligned NB (block size) candidates:

```
96, 128, 160, 192, 256, 320, 384, 512,
1024
```

By fixing the problem size at $N = 10240$ or $N = 20480$, we evaluated these parameters and recorded the resulting performance metrics, as depicted in Fig. 3.

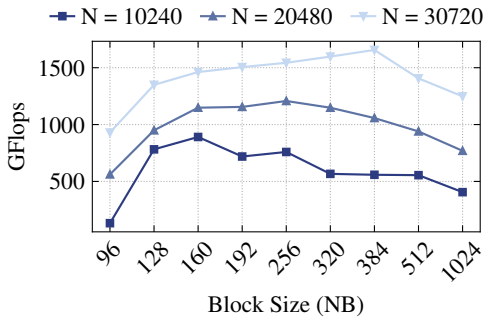


Fig. 3. HPL performance under different block sizes (NB)

From these evaluations, we narrowed down our selection to the three optimal `NB` candidates: 160, 256, and 384.

Subsequently, we benchmarked various $P \times Q$ 2D process grid topologies. The performance outcomes of these grid layouts are illustrated in Fig. 4.

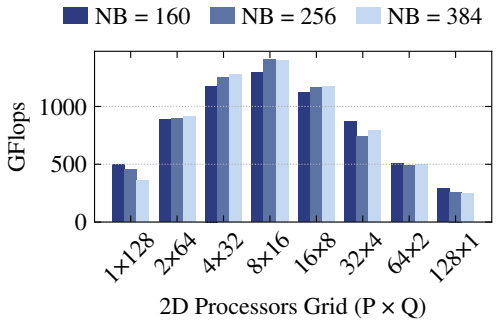


Fig. 4. HPL performance under different process grid layouts (P×Q)

Based on this analysis, the 8×16 layout emerged as the most suitable process grid configuration ($P \times Q = 8 \times 16$).

Finally, constrained by the system's total memory capacity (512 GB), we tested a series of problem sizes (N). As detailed in Fig. 5, the system ultimately achieved a sustained peak performance of **4283.1** GFLOPS (see Fig. 6).

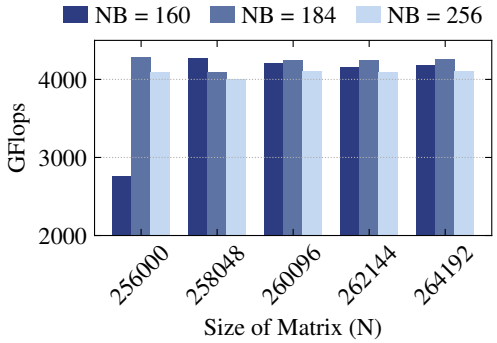


Fig. 5. HPL performance under different problem sizes

```
================================================================================
T/V                N    NB     P     Q               Time                 Gflops
--------------------------------------------------------------------------------
WR11C2C4      256000   184     8    16            2611.39              4.2831e+03
HPL_pdgesv() start time Mon Mar  2 05:02:52 2026

HPL_pdgesv() end time   Mon Mar  2 05:46:24 2026

--------------------------------------------------------------------------------
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)=   7.24692320e-04 ..... PASSED
================================================================================
```

Fig. 6. Best achieved performance of HPL on CPU node

For the CPU, we referred to the AMD website and found that the AMD EPYC 9K84 has a frequency of 2.6 GHz with 96 cores per socket, comprising three nodes in total. In our actual testing, we utilized 128 processes. Therefore, the theoretical performance of the CPU is :

$$2.6\ \text{GHZ} \times 128\ \text{Cores} \times 16\ \text{flop} = 5324.8\ \text{GFLOPS}$$

Computational efficiency can be calculated using the following method:

$$\frac{4283.1}{5324.8} \times 100\% = 80.4\%$$

Ultimately, the computational efficiency meets our expected requirements, providing a perfect reference for subsequent challenge tasks.

F. HPL on GPU

Employing a similar methodology, we conducted HPL performance benchmarking on the GPU node (8 × A100). As illustrated in , the system achieved a final sustained performance of **279.8** TFLOPS and the LU is **558.3** TFLOPS.

```
====================== HPL MxP Result=========================

   EPS       ..................       =   2.000000E-16
   Threshold    ..................       =   1.600000E+01
||Ax-b||_oo   ..................       =   6.106227E-14
||A  ||_oo    ..................       =   1.810819E+05
||x  ||_oo    ..................       =   1.115533E-05
||b  ||_oo    ..................       =   9.999752E-01
||Ax-b||_oo / (EPS * (||A||_oo * ||x||_oo + ||b||_oo) * N) =   5.616465E-04 ..... PASSED

  N = 180000, NB = 288, NPROW = 2, NPCOL = 4, SLOPPY-TYPE = 2
    GFLOPS = 2.8651e+05, per GPU =   35813.13
  LU GFLOPS = 5.7166e+05, per GPU =   71458.10

====================== HPL MxP Result=========================
```

Fig. 7. Best achieved performance of HPL on GPU node

G. HPCG Performance Tuning Method

We utilized HPCG 3.0 to benchmark the CPU performance, while employing the NVIDIA GPU binaries for HPCG 3.1 (optimized for the Ampere architecture) to evaluate the GPU performance. The peak performance achieved was **1748.79** GFLOPS in GPU node. Fig. 8.

The High Performance Conjugate Gradients (HPCG) benchmark is designed to complement the High Performance LINPACK (HPL) benchmark by more closely mirroring the computational and data access patterns of a broad and diverse range of critical applications. Compared to HPL, the differential equation computations involved in HPCG are more complex. It evaluates massive irregular memory accesses and imposes stringent demands on high memory capacity and low-latency networks, making it a far more practical metric for real-world workloads.

H. HPCG on CPU

During CPU-based HPCG benchmarking, the selection of the problem size has a decisive impact on the validity and interpretability of the results.

According to the HPCG design specifications, the minimum problem size for CPU evaluations is 24, and it must be a multiple of 8 to satisfy the constraints of grid partitioning and parallel computation algorithms. To ensure the benchmark accurately reflects the system's true computational capability, the problem size must significantly exceed the processor's cache capacity. Otherwise, the majority of the data will reside in the high-speed cache, yielding results that disproportionately benefit from high cache hit rates and deviate from real-world application scenarios. Dictated by this principle, the problem size should consume at least $\frac{1}{4}$ of the total main memory.

Building upon this, we selected multiple sets of problem sizes for our evaluations and uniformly fixed the execution time to 1800 seconds per run to guarantee the stability of the performance metrics. Ultimately, the peak performance we obtained was **73.67** GFLOPS, as shown in Fig. 8.

```
GB/s Summary=
GB/s Summary::Raw Read B/W=453.916
GB/s Summary::Raw Write B/W=104.903
GB/s Summary::Raw Total B/W=558.819
GB/s Summary::Total with convergence and optimization phase overhead=549.718
GFLOP/s Summary=
GFLOP/s Summary::Raw DDOT=37.3918
GFLOP/s Summary::Raw WAXPBY=115.723
GFLOP/s Summary::Raw SpMV=68.6604
GFLOP/s Summary::Raw MG=75.5778
GFLOP/s Summary::Raw Total=73.6657
GFLOP/s Summary::Total with convergence overhead=73.6657
GFLOP/s Summary::Total with convergence and optimization phase overhead=72.466
User Optimization Overheads=
User Optimization Overheads::Optimization phase time (sec)=1.93e-07
User Optimization Overheads::Optimization phase time vs reference SpMV+MG time=1.9328e-05
DDOT Timing Variations=
DDOT Timing Variations::Min DDOT MPI_Allreduce time=41.4552
DDOT Timing Variations::Max DDOT MPI_Allreduce time=80.0833
DDOT Timing Variations::Avg DDOT MPI_Allreduce time=64.0042
Final Summary=
Final Summary::HPCG result is VALID with a GFLOP/s rating of=72.466
Final Summary::HPCG 2.4 rating for historical reasons is=73.6657
```

Fig. 8. Best achieved performance of HPCG on CPU node

The global computational domain of HPCG is constructed by scaling the local computational domains via the number of MPI processes. This relationship is expressed as:

$$(\text{NR} \times N_x) \times \big(\text{NR} \times N_y\big) \times (\text{NR} \times N_z)$$

where $N_x = N_y = N_z$, and NR represents the number of MPI processes. To evaluate the impact of different problem dimensions on the experimental outcomes, we first fixed the MPI process count at 64. This ensures that performance variations across different configurations solely reflect the impact of the problem size.

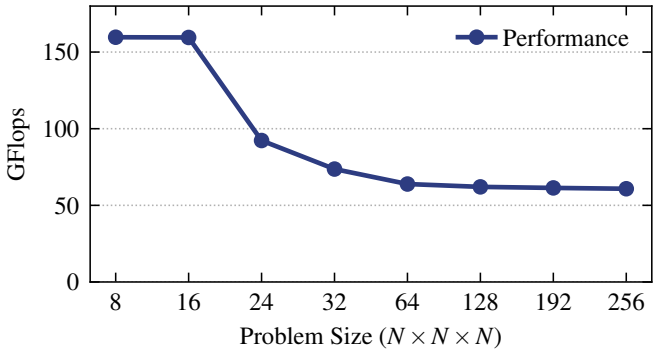


Fig. 9. Different Size Comparison

As can be easily seen from the trends shown in Fig. 9, the smaller the problem size, the faster the execution. This is because a smaller scale allows data to be stored in the cache, which accelerates data access and improves computational speed. Among the sets of 8, 16, 24, 32, 64, 128, 192 and 256, excluding the first three sets where the scale is too small, the best result of 73.67 GFlops was achieved with a data size of $32 \times 32 \times 32$. All tests were allocated across 64 processes, with each process containing a single thread.

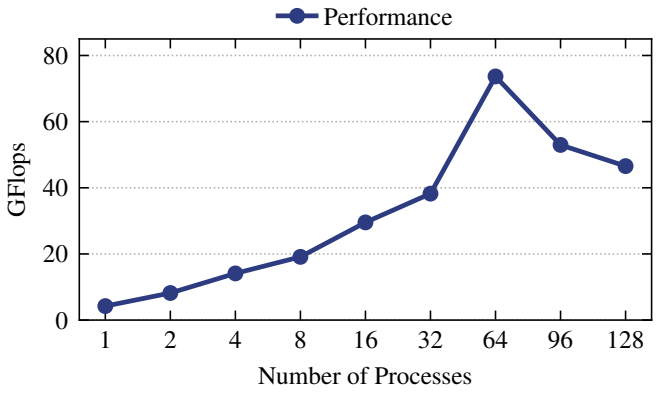


Fig. 10. Different Process Comparison

Having established the optimal problem size, we proceeded to investigate the correlation between the MPI process count and overall performance. As illustrated in Fig. 10, under a fixed matrix size, increasing the number of MPI processes initially yields superior performance compared to using fewer processes. However, beyond a certain threshold of MPI ranks, performance begins to degrade, resulting in an overall trend that exhibits a distinct peak.

I. HPCG on GPU

Using a similar method, we conducted an HPCG performance test on an 8 x A100 GPU node. The final result achieved, Fig. 11, was **1748.79** GFLOPS.

```
GB/s Summary=
GB/s Summary::Raw Read B/W=12365.2
GB/s Summary::Raw Write B/W=2858.03
GB/s Summary::Raw Total B/W=15223.3
GB/s Summary::Total with convergence and optimization phase overhead=13261.9
GFLOP/s Summary=
GFLOP/s Summary::Raw DDOT=1250.93
GFLOP/s Summary::Raw WAXPBY=900.737
GFLOP/s Summary::Raw SpMV=1664.03
GFLOP/s Summary::Raw MG=2165.43
GFLOP/s Summary::Raw Total=2007.43
GFLOP/s Summary::Total with convergence overhead=1858.73
GFLOP/s Summary::Total with convergence and optimization phase overhead=1748.79
User Optimization Overheads=
User Optimization Overheads::Optimization phase time (sec)=0.750842
User Optimization Overheads::Optimization phase time vs reference SpMV+MG time=0.259746
DDOT Timing Variations=
DDOT Timing Variations::Min DDOT MPI_Allreduce time=5.74777
DDOT Timing Variations::Max DDOT MPI_Allreduce time=7.67311
DDOT Timing Variations::Avg DDOT MPI_Allreduce time=6.94454
Final Summary=
Final Summary::HPCG result is VALID with a GFLOP/s rating of=1748.79
Final Summary::HPCG 2.4 rating for historical reasons is=1760.73
```

Fig. 11. Best achieved performance of HPCG on GPU node

The HPCG benchmark on the A100 GPU achieved a validated rating of 1748.79 GFLOP/s , supported by a raw total bandwidth of 15223.3 GB/s with a dominant read component of 12365.2 GB/s. Among the kernels, MG delivered the highest performance at 2165.43 GFLOP/s , while the optimization overhead remained minimal at 0.751 seconds, demonstrating excellent computational efficiency.

V. Embodied World Model Optimization

A. Overview

In the Embodied World Model Optimization track, the objective is to accelerate the execution time of UnifoLM-WMA-0 [3] on the provided dataset. We profile and analyze the overall model inference process. By optimizing the code structure and memory layout, and employing quantization, caching, and parallelization, we achieve a maximum speedup of $\mathbf{7.58}\times$ while preserving the final video generation quality.

a) *Dataset:*

The dataset consists of five scenarios, each containing four cases, yielding a total of twenty samples.

b) *Model Analysis:*

The model structure is illustrated in Fig. 12. The inference process primarily consists of a decision-making mode and a simulation mode. During a single inference pass, the model first encodes the input historical images, states, actions, and given instructions into features. It then enters the decision-making mode, feeding the encoded features into WMAModel and Action/State UNet, which are distinct instances of `ConditionalUnet1D` for sampling, which updates the generated actions with the historical action vector. Subsequently, the model re-encodes the features and enters the simulation mode, generating new state vectors and video sequences through sampling. These update the historical states and image sequences. Through the iterative execution of this process, the model ultimately generates the complete predicted video.

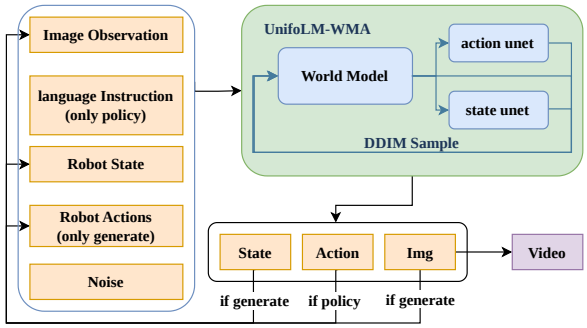


Fig. 12. Original structure diagram

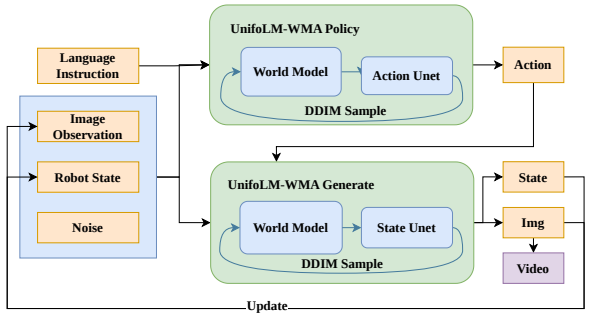


Fig. 13. Optimized structure diagram

B. Environment

The hardware resource is shown in Table VII while the software resource is shown in Table VIII.

TABLE VII
HARDWARE ENVIRONMENT

Device	Detail
CPU	Hygon C86-3G 7380 32-core Processor @ 2.2GHz × 2
GPU	NVIDIA A800 80GB PCIe × 2

TABLE VIII
SOFTWARE ENVIRONMENT

Description	Version
Operation System	Ubuntu-22.02
Language	Python-3.10.18
Framework	torch-2.3.1
CUDA	Version 13.1

C. Structural Optimization

Model analysis reveals that strong coupling between the code for the decision-making mode and simulation mode, with no clear separation achieved. The workflow in the original codebase contains significant redundancy. Consequently, we introduce a series of optimizations to the overall workflow.

a) *Pruning Redundant UNet for Single-Pass Decoding:*

In the original inference logic, both the decision-making mode and the simulation mode execute the State UNet and Action UNet within the WMAModel. Because the decision-making mode only needs to output actions and the simulation mode only needs to output states, we optimize the inference logic to remove this redundancy, ensuring that each mode only runs the UNet module corresponding to its function. Furthermore, video generation is only a necessary output in the simulation mode. The original logic performs video generation concurrently with action generation in the decision-making mode, yet subsequent processes do not utilize this video. To improve inference efficiency, we eliminate the video generation step in the decision-making mode. The optimized inference workflow is shown in Fig. 13.

b) *Memory Copy Elimination:*

During the model inference process, analysis indicates that the original code contains redundant memory copy operations. For instance, in the `efficient_forward` function of the `CrossAttention` module, the original attention computation involves reshaping the dimensions from (B, N, H, D) to (B×H, N, D). This operation incurs unnecessary memory copy overhead. Because native FlashAttention provides strong support for multi-head attention, we eliminate this dimension transformation and its associated memory copies.

c) *Asynchronous Loading and Saving:*

To address I/O bottlenecks throughout the model lifecycle, this work introduces systematic optimizations for both weight loading and result persistence. In the initialization phase, we first restructure the underlying data serialization by migrating from traditional pickle-based checkpoints to a zero-copy memory-mapped format (safetensors). This transition eliminates the inherent deserialization overhead and redundant RAM allocations. Building upon this contiguous memory layout, we propose an asynchronous prefetching strategy that parallelizes disk I/O operations with the instantiation logic of the Diffusion model. By decoupling data transfer from model initialization, the synergy between zero-copy loading and asynchronous scheduling achieves deep latency hiding.

Regarding result saving, a multi-threaded immediate saving mechanism addresses the calculation stalls caused by conventional serial storage logic. By triggering a background write thread immediately after a video frame is generated, the system allows the computation of subsequent frames to overlap with the I/O of the current frame, which substantially reduces the time overhead for data persistence.

d) *Skip Init:*

Code analysis indicates that model instantiation accounts for a significant portion of the execution time. Detailed profiling reveals that the model performs random parameter initialization using methods such as kaiming_normal_ and xavier_uniform_ during this stage, although these values are subsequently overwritten by the loaded weight files. Since this initial process is redundant, bypassing the random initialization phase substantially accelerates model instantiation.

e) *Selective 16-bit Quantization:*

The baseline code operates entirely in a float32 environment. Given that the competition rules prohibit quantization methods below 16 bits, we primarily employ float16 during the inference stage. Experiments demonstrate that converting all model parameters to float16 results in significant accuracy loss. Further investigation reveals that the parameters of the noise scheduler (specifically, `alpha` and `alphas_cumprod_prev`) are highly sensitive to precision. Even marginal quantization errors in these critical scheduling parameters compound multiplicatively across the iterative denoising steps, leading to severe distribution shifts in the sampled trajectory. By retaining these parameters in float32, we effectively maintain the stability of the model accuracy without compromising the acceleration performance.

D. Caching Mechanism

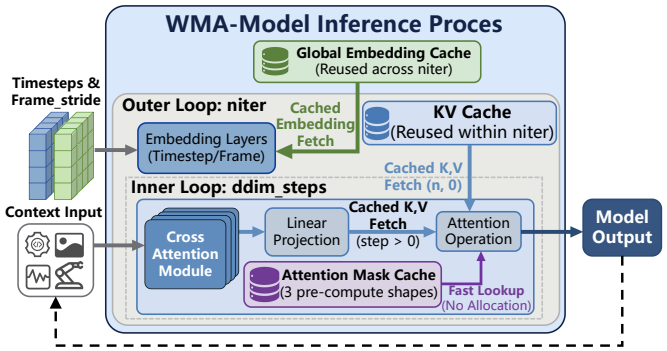


Fig. 14. Multi-level cache diagram

Redundant computations exist during the model inference process. We introduce a multi-level caching mechanism to reuse intermediate results, which significantly reduces inference latency. The overall caching architecture is illustrated in Fig. 14.

a) *KV Cache:*

The first optimization targets the forward pass of `CrossAttention` within the WMAModel. Experiments reveal that the system computes the KV matrices using the context when the input context is not empty. During different ddim_steps within the same niter, the model shares an identical context. Consequently, we generate and cache the KV matrices exclusively during the initial computation. Subsequent steps directly reuse these cached results, effectively bypassing repetitive computations.

b) *Attention Mask Cache:*

Similarly, during the forward pass of `CrossAttention`, the system generates an `attention_mask` for the action component within the context to perform masked attention computation. Because the tensor size of such masks is large, frequent generation incurs substantial cudaMalloc overhead and increases latency, as depicted in Fig. 15. Given that the mask generation solely depends on the shape of the input tensor, and only three distinct mask shapes exist throughout the entire inference process, we cache the masks to optimize performance.

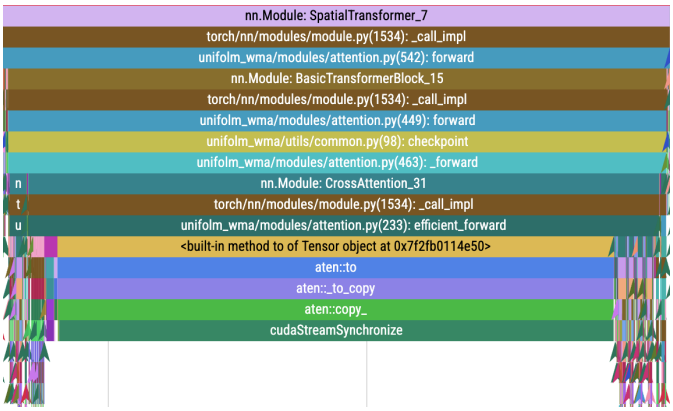


Fig. 15. Flame graph of the Spatial Transformer

c) *Embedding Cache:*

During the forward pass of the WMAModel, the system processes embeddings for timesteps and frame_stride. Although the embedding results differ within a single niter because each ddim_step corresponds to a distinct timestep, these embedding computations repeat across different niters. The code demonstrate that different niters execute identical calculation logic for timestep and frame_stride embeddings. Therefore, reusing these embedding results effectively prevents redundant computations across different niters.

E. Parallelization

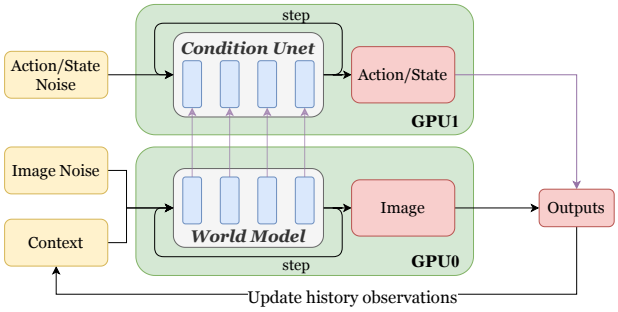


Fig. 16. Parallelization of World Model and Condition U-Net

In the original inference pipeline, the WMAModel employs a sequential execution logic. A strong sequential dependency exists between the World Model and the Action/State UNet, where the latter must wait for the former to output intermediate latent states before execution. However, an in-depth analysis of the network topology reveals that this dependency is fundamentally a fine-grained coupling at the layer level.

Based on this observation, we propose a pipeline-like parallel inference mechanism, as illustrated in Fig. 16 [4]. We deploy the World Model and the Action/State UNet on separate GPU nodes. By establishing layer-synchronized

communication primitives, the Action/State UNet initiate computation immediately upon receiving the feature states from specific layers of the World Model. This enables the overlapping of computational tasks in the temporal dimension. This design effectively eliminates the sequential bottleneck and significantly improves the inference throughput of the system.

F. Kernel Optimization

During the model inference process, we improve inference efficiency by employing high-performance operators and developing custom fused operators.

a) *GEGLU Operator Fusion:*

The FeedForward module in model inference involves the GEGLU operator. Its standard procedure chunks the tensor output from the linear layer, applies the GELU function to one chunk, and subsequently performs element-wise multiplication with the other chunk. Analysis reveals that this process requires multiple global memory accesses, including independent GELU computations and multiplication operations. We develop a custom fused operator that merges tensor chunking, GELU computation, and element-wise multiplication into a single GPU kernel. This approach eliminates the allocation and memory access overhead of intermediate tensors. Concurrently, utilizing sixteen-byte vectorized memory access through int4 type casting ensures that a single memory transaction reaches the maximum throughput of the hardware bus. This optimization strategy significantly increases arithmetic intensity and achieves substantial acceleration compared to the non-fused operator implementation.

b) *GroupNorm and SiLU Fusion:*

Algorithm 1: Welford's Online Algorithm[5]

Require: State 1 $(\text{mean}_1, \text{m2}_1, w_1)$, State 2 $(\text{mean}_2, \text{m2}_2, w_2)$

Ensure: Merged State $(\text{mean}, \text{m2}, w)$

Compute weight sum: $w \leftarrow w_1 + w_2$

Compute mean difference: $\Delta \leftarrow \text{mean}_2 - \text{mean}_1$

Update global mean: $\text{mean} \leftarrow \text{mean}_1 + \Delta \cdot \left(\frac{w_2}{w}\right)$

Update sum of squared deviations: $\text{m2} \leftarrow \text{m2}_1 + \text{m2}_2 + \Delta^2 \cdot w_1 \cdot \left(\frac{w_2}{w}\right)$

Return $(\text{mean}, \text{m2}, w)$

We observe that the ResBlock within the WMA-Model contains numerous GroupNorm and SiLU operations. Because the GPU utilization of these operators is relatively low, we perform operator fusion on them. The baseline implementation typically computes $y = \text{group_norm}(x; G, \gamma, \beta, \varepsilon)$ and subsequently generates the output as $\text{out} = \text{SiLU}(y)$. The performance bottlenecks of this approach typically stem from the bandwidth overhead of writing back and reading the intermediate result y to and from global memory at least once. Additional bottlenecks include the kernel launch and scheduling overhead, which is particularly significant for small tensors or fragmented shapes. To address the characteristics of the baseline, we integrate the activation function directly into the application phase of GroupNorm. This involves calculating the statistical mean and reciprocal standard deviation for normalization, applying the affine transformation, immediately executing the activation, and writing back the output. This approach prevents the intermediate round-trip of writing the GroupNorm output and reading it back for activation. The specific workflow is as follows.

Algorithm 2: Fused GroupNorm and SiLU (Channels-Last)

Require: Input tensor $X \in \mathbb{R}^{N \times H \times W \times C}$ in HBM, Learnable parameters $\gamma, \beta \in \mathbb{R}^C$, Groups G, Epsilon ε

Ensure: Output tensor $O \in \mathbb{R}^{N \times H \times W \times C}$

// Phase 1: Collect Statistics (Kernel 1)

for each batch $n \in [0, N)$ and group $g \in [0, G)$ **do**

  Initialize $\text{mean}_{n,g} \leftarrow 0$, $\text{m2}_{n,g} \leftarrow 0$, weight $\leftarrow 0$ in on-chip SRAM

  for each spatial offset $s \in [0, H \times W)$ **do**

    Load channel vector $X_{n,s,g}$ from HBM to SRAM

    Update Welford statistics $(\text{mean}_{n,g}, \text{m2}_{n,g})$ using $X_{n,s,g}$

  end

  Compute $\text{var}_{n,g} \leftarrow \frac{\text{m2}_{n,g}}{\text{weight}}$

  Compute $\text{rstd}_{n,g} \leftarrow \frac{1}{\sqrt{\text{var}_{n,g} + \varepsilon}}$

  Write $\text{mean}_{n,g}$ and $\text{rstd}_{n,g}$ to HBM

end

// Phase 2: Apply and Activate (Kernel 2)

for each batch $n \in [0, N)$ and spatial block idx **do**

  for each channel $c \in C_G$ **do**

    Load $\text{mean}_{n,g(c)}$ and $\text{rstd}_{n,g(c)}$ from HBM to SRAM

    On chip, compute affine terms: $a \leftarrow \text{rstd}_{n,g(c)} \cdot \gamma_c$, $b \leftarrow \beta_c - a \cdot \text{mean}_{n,g(c)}$

    Load input vector $X_{n,\text{ idx},c}$ from HBM

    Compute $Z \leftarrow a \cdot X_{n,\text{ idx},c} + b$

    Compute activation: $O_{n,\text{ idx},c} \leftarrow Z \cdot \left(\frac{1}{1+\exp(-Z)}\right)$

    Write $O_{n,\text{ idx},c}$ to HBM

  end

end

c) *xFormers:*

The implementation of the CrossAttention class involves two forward propagation methods. The conventional forward

method employs standard Einstein summation to compute cross-attention, whereas the efficient forward method utilizes the memory_efficient_attention operator provided by the xformers library [6], [7]. Furthermore, within classes such as SATokenProjector and Resampler, the PerceiverAttention module also relies on Einstein summation to implement the attention mechanism. The aforementioned functions based on general matrix multiplication can all be replaced by the memory_efficient_attention operator from the xformers library to achieve more efficient attention computation.

G. Selective DeepCache Mechanism

To further accelerate the sampling process, we integrated DeepCache [8], a training-free paradigm designed to expedite diffusion models. DeepCache structurally exploits the temporal redundancy inherent in the sequential denoising process.

a) *Theoretical Foundation:*

Observations of diffusion models reveal that high-level features located in the deeper layers of the U-Net evolve sluggishly across adjacent timesteps, whereas low-level features in the shallow layers fluctuate rapidly. DeepCache capitalizes on this property by decoupling the U-Net architecture into dynamic shallow blocks and cached deep blocks. The process is shown in Fig. 17.

During the inference phase, the system caches the high-dimensional feature maps from the deep encoder blocks and the bottleneck layer at a given timestep. For subsequent timesteps within a predefined `deepcache_interval`, the forward pass entirely bypasses these computationally expensive deep layers. Instead, it directly routes the cached feature maps to their corresponding decoder layers via the U-Net's long skip connections. This architectural shortcut drastically prunes the Floating Point Operations (FLOPs) required per denoising step without requiring model retraining.

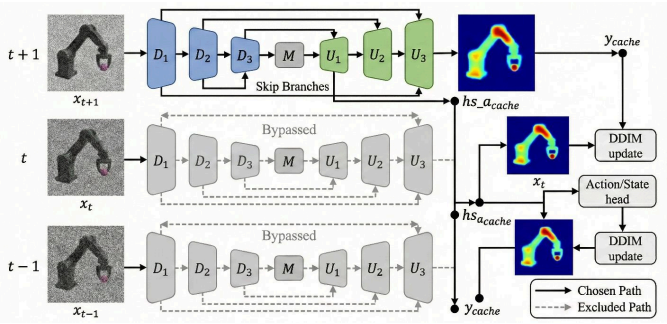


Fig. 17. DeepCache

b) *Case-Specific Adaptation:*

Although DeepCache significantly reduces computational latency, approximating deep features renders it an inherently lossy optimization. Empirical evaluations demonstrate that a global application of DeepCache degrades the visual fidelity of generated video sequences, occasionally causing the Peak Signal-to-Noise Ratio (PSNR) to fall below the strict precision and accuracy thresholds of the competition. To balance inference throughput with structural integrity, we implement a selective DeepCache strategy.

In terms of selective activation, we profile the baseline PSNR for twenty samples across five scenarios. DeepCache is exclusively enabled for robust cases where the baseline PSNR provides a sufficient margin to accommodate the quality degradation induced by caching. For these activated cases, we rigorously tune two critical hyperparameters to maintain the PSNR constraint: `deepcache_warmup` and `deepcache_interval`. The `deepcache_warmup` parameter specifies the number of initial denoising steps executed natively without caching. Since the early stages of the diffusion process establish the foundational geometry and low-frequency structure of the generated frames, performing these steps with full computation prevents severe macroscopic distortions. The `deepcache_interval` parameter denotes the frequency at which the cached deep feature maps are refreshed. While a larger interval yields higher speedups, it also exacerbates quality degradation. We calibrate this parameter for each case to identify the optimal Pareto frontier between execution speed and PSNR. By restricting this lossy acceleration to robust cases and dynamically tuning the caching intervals, we achieve substantial additional inference speedups while strictly adhering to the numerical constraints of the competition.

H. Conclusion

a) *Analysis:*

Following a series of optimizations, the inference speed of UnifoLM-WMA-0 improves significantly. Tests conducted on the NVIDIA GPU platform demonstrate an overall speedup ratio ranging from **3.72x** to **7.58x** compared to the baseline, as illustrated in Fig. 18. Beyond enhancing the inference speed, we rigorously evaluate the confidence metrics of the model outputs to ensure the optimizations do not compromise prediction accuracy. Table IX summarizes the overall experimental results.

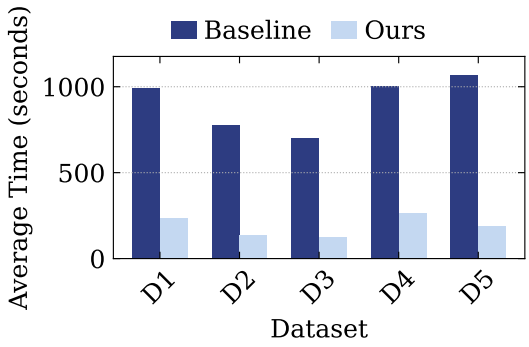


Fig. 18. Comparison of average execution time (in seconds) between the baseline and our method. The dataset abbreviations used in the x-axis correspond to the following original datasets: D1: unitree_g1_pack_camera, D2: unitree_z1_dual_arm_cleanup_pencils, D3: unitree_z1_dual_arm_stackbox, D4: unitree_z1_dual_arm_stackbox_v2, D5: unitree_z1_stackbox.

TABLE IX

PERFORMANCE COMPARISON OF OUR METHOD AGAINST THE BASELINE. CASES MARKED WITH AN ASTERISK (*) INDICATE THAT DEEPCACHE WAS NOT USED.

Dataset	Case	PSNR	Time (s) ↓		
			Baseline	Ours	Speedup
Z1 Stack Box	1	25.41	1069.94	165.60	**6.46×**
	2	27.28	1068.38	165.06	**6.47×**
	3	25.15	1062.12	206.33	**5.15×**
	4	25.78	1076.68	208.57	**5.16×**
Z1 Dual Arm Stack Box	1	26.50	707.11	155.19	**4.56×**
	2	25.16	696.99	123.49	**5.64×**
	3	26.14	704.62	92.91	**7.58×**
	4	28.89	698.58	121.47	**5.75×**
Z1 Dual Arm Stack Box v2	1*	25.68	1004.44	265.71	**3.78×**
	2*	29.14	1001.91	264.05	**3.79×**
	3*	27.06	1006.53	264.43	**3.81×**
	4*	26.01	1003.48	269.48	**3.72×**
Z1 Dual Arm Cleanup Pencils	1	26.57	762.73	136.92	**5.57×**
	2	25.38	771.74	132.86	**5.81×**
	3	26.28	807.71	144.47	**5.59×**
	4	27.95	776.10	134.87	**5.75×**
G1 Pack Camera	1	26.67	994.31	207.14	**4.80×**
	2	25.02	994.29	215.53	**4.61×**
	3*	29.90	992.74	264.42	**3.75×**
	4*	32.05	993.22	266.56	**3.73×**

b) *Reproduction:*

For specific steps, please refer to the **README** in the project file.

VI. NUMERICAL RELATIVITY CODE AMSS-NCKU OPTIMIZATION

A. Overview

In this challenge, we traced and analyzed the complete execution flow of the hotspot programs, `ABE` and `TwoPunctureABE`, within the workflow. By employing a variety of methods—including translating Fortran functions to C++, optimizing memory access, batching critical functions, utilizing sliding windows to reuse intermediate variables, applying SIMD vectorization, and utilizing hybrid OpenMP programming—we achieved a $\mathbf{4.33}\times$ speedup and successfully passed the 1% precision test.

Additionally, we addressed and fixed existing bugs in the original GPU version. Building upon the baseline GPU implementation, we rewrote the core kernels, ultimately outperforming the original GPU version by $\mathbf{8.3}\times$.

B. Environment

The hardware resource is shown in Table VII while the software resource is shown in Table VIII.

TABLE X

HARDWARE ENVIRONMENT

Device	Detail
CPU	2 × AMD EPYC 9K84 @ 2.6 GHz
GPU	NVIDIA H100 80GB PCIE × 4

TABLE XI

SOFTWARE ENVIRONMENT

Description	Version
Operation System	Ubuntu-25.04
Language	15.2.1 20260103 (GCC)

C. Baseline & Analysis

a) *Baseline Flame Graph:*

On the our cluster, we evaluated the baseline performance using 64 MPI processes. The baseline execution time for AMSS-NCKU was 32530.6s, and for `TwoPunctureABE` it was 455.86s. The execution time of the `ABE` program significantly dominated that of `TwoPunctureABE`. Consequently, our optimization efforts were heavily focused on `ABE`, with minor optimizations applied to `TwoPunctureABE`. For `ABE`, the profiling [9] results sampled from a single process are illustrated in Fig. 19.

b) *Code Structure Analysis:*

The `ABE` program simulates the evolution of black hole positions over discrete `Timesteps`. During each `Timestep`, the full field changes are computed, which subsequently

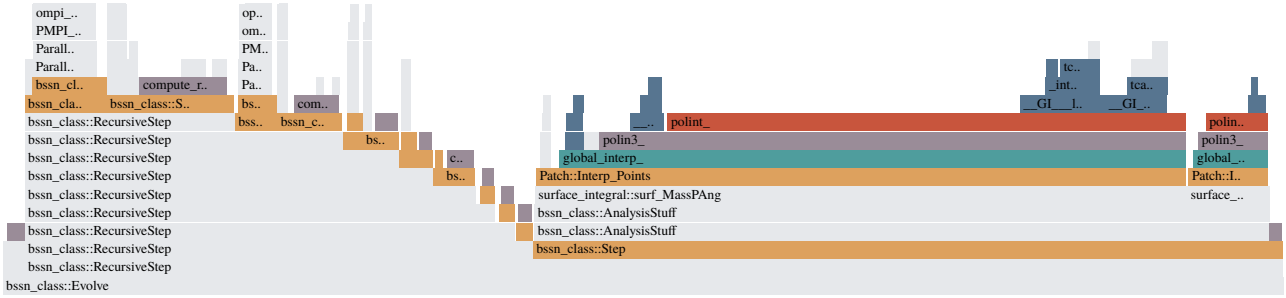


Fig. 19. Flame graph of the original `ABE` implementation

updates the black hole positional information at the finest resolution level.

The evolution is globally orchestrated by the `ADM` object. The overall workflow is depicted in Fig. 20.

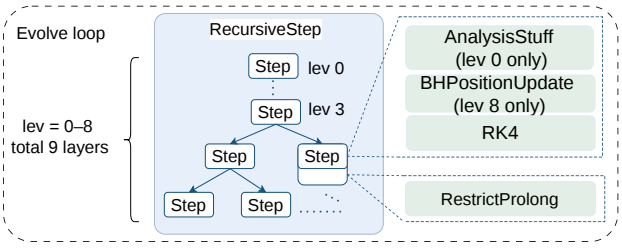


Fig. 20. ADM Spatiotemporal Evolution Workflow

Each `Timestep` invokes the `Evolve` simulation. Within `Evolve`, the simulation is divided into 9 levels, where each level represents a specific temporal resolution. Concurrently, the `Step` function is utilized within each level to simulate the corresponding physical processes.

Within the `ADM` object, some critical physical data are stored in a `Patch` linked list inside the `cgh` class. Each `Patch` object contains pointers to a `Block` linked list, where the `Block` holds the fundamental data arrays. The `Step` function drives the black hole simulation by updating the basic physical states residing in these `Block`.

D. Profiling & Analysis

By profiling the various code modules of the `ABE` program, we obtained the time breakdown across different simulation Levels within a single Timestep, as shown in the table below:

TABLE XII
TIME BREAKDOWN OF A TIMESTEP[1]

Period	Parts	Cost	Total
Level 0	AnalysisStuff	7.621s	9.882s
	RestrictProlong	0.000s	
	compute_rhs_bssn	0.145s	
Level 1-7	AnalysisStuff	0.000s	3.379s
	RestrictProlong	1.047s	
	compute_rhs_bssn	1.991s	
Level 8	AnalysisStuff	0.000s	2.811s
	RestrictProlong	0.908s	
	compute_rhs_bssn	1.243s	

As observed, `Level0` consumes 9.882s and `Level8` consumes 2.811s, taking up 82.5% of the total timestep duration. Specifically, `AnalysisStuff` is the bottleneck in Level 0, while `compute_rhs_bssn` dominates `Level8`. For the remaining levels, execution time is roughly split between `RestrictProlong` and `compute_rhs_bssn`. These three components were identified as the primary performance bottlenecks.

Further drill-down analysis revealed the top three most time-consuming Fortran functions:

- `global_interp` , invoked in `AnalysisStuff`, accounting for 48.4% of total execution time.
- `compute_rhs_bssn` function, invoked in `Step`, accounting for 18.0%.
- `prolong3` function, invoked in `RestrictProlong`, accounting for 13.6%.

To further analyze these hotspots, we captured the CPU performance metrics during a single execution of the hotspot functions, as detailed in Table XIII.

TABLE XIII
PERFORMANCE METRICS OF BOTTLENECKS

Function Name	Metrics ($\times 10^4$ Cycles)				
	Stalls L1 Miss	Stalls L2 Miss	Stalls L3 Miss	Bus Cycles	Total
global_interp	21.71	10.90	10.89	3.59	4.87
compute_rhs	133055	48518.8	48518.8	13463.5	18275.0
prolong3	49.42	17.56	17.56	7.99	10.56

This analysis yielded several critical observations:

- The vast majority of CPU cycles are stalled waiting for memory access, evidenced by the `bus_cycle` metric comprising nearly 50% of total cycles.
- The massive CPU stall times are caused by severe cache thrashing. The roughly equivalent stall counts across L1, L2, and L3 misses indicate that data fetches frequently penetrate the entire cache hierarchy, fetching directly from main memory.

Based on this, an in-depth code review of the three hotspots concluded:

1) For `prolong3`, the profiling with gporf showed that the core computational bottleneck lies within a repeated 3D convolution process: `convolve_x` takes 76.99%, `convolve_y` takes 12.47%, and `convolve_z` takes 6.29%.
2) For `compute_rhs_bssn`, memory access bottlenecks were distributed somewhat evenly, with no isolated hotspots. However, the logical structure contained massive redundant numerical data movements across memory sweeps, leading to poor efficiency.
3) For `global_interp`, the bottleneck is exclusively memory-bound, making it highly suitable for memory-access optimizations.

[1]The sum of the data in this table differs from Table XIV due to timing errors caused by checkpoints. We consistently selected the shorter durations for the baseline to ensure that our speedup results are not overstated, maintaining the scientific integrity of the comparison.

Furthermore, a statistical analysis of execution times across different MPI processes revealed a severe load imbalance in `Analysi sStuff` and `prolong3`, as shown in Fig. 21.

Most processes complete within $0 \sim 0.3$ seconds, while a minority of processes take up to 8 seconds. This imbalance causes significant synchronization wait times inside the hotspot functions, leaving many CPU cores idling and reducing overall compute efficiency.

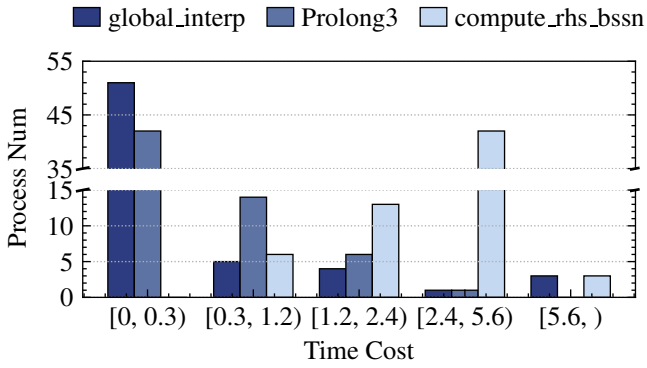


Fig. 21. The workload distribution across different MPI processes

E. Optimization

To address the aforementioned issues, we implemented several targeted optimizations.

a) *Use MPI-OpenMP Hybrid Programming for Load Balancing:*

To address the MPI load imbalance, we identified that the root cause is the static workload distribution based on process rank during initialization. Consequently, some processes are assigned `Patch` containing a massive number of `Block` to compute, while others are allocated virtually no `Patch` at all.

To mitigate this, we applied an MPI+OpenMP hybrid programming model. By parallelizing the computation inside the bottleneck functions using OpenMP, we utilized the idling CPU cores to process the workload. This reduced the maximum computation time and improved parallel efficiency.

After applying this optimization, we benchmarked a typical workload, and the resulting speedup is shown in Fig. 22.

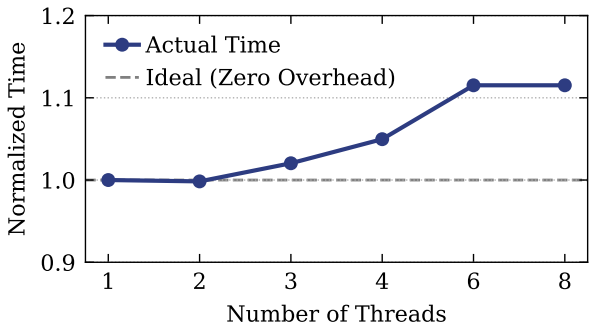


Fig. 22. Speedup scaling of the bottleneck using OpenMP threads

We observed that the speedup saturates at 6 threads, beyond which adding more threads yields diminishing returns.

b) *Batched Processing and Memory Locality:*

To tackle the high cache miss rate, we primarily optimized the `Interp_Point` function, which consumes the largest portion of execution time.

Code analysis of `Interp_Point` revealed an outer loop iterating `NN` times, and an inner loop traversing `Block` and calling `global_interp`. Inside `global_interp`, the program reads `fgfs` rows from the block, assembles a `ya` data cube based on a `pox` index, and calls `polint` to compute the interpolation. This is illustrated in Listing 1.

```
// in Function Interp_points
for (i in range(0, NN)) {
  for (Block) {
    { // global_interp
      read pox, fgfs in block
      ya = fgfs[pox]
      polint(ya)
    }
  }
}
```

Listing 1. Original logic in `Interp_points`

We discovered that the total number of Blocks is substantially smaller than `NN`. The iterations over `NN` are independent, yet each `NN` iteration repeatedly accesses the same `fgfs` rows within a Block. Because the memory stride of the `fgfs` variables is massive, the original execution order caused severe cache evictions and redundant data fetches.

In a multi-threaded environment, concurrent calls to `global_interp` access `fgfs` arrays across different Blocks, causing the working set size to far exceed the cache capacity, leading to frequent cache misses.

To rectify this, we applied the following transformations:

1) Loop Interchange: We swapped the `NN` loop with the `Block` loop.
2) Data Packing: Modified global_interp to assemble multiple ya cubes across several `NN` loops simultaneously.
3) Batched Interpolation: Modified polint to process multiple ya inputs simultaneously.

During the loop interchange process, we pre-computed a mapping from the `pox` indices in the `NN` iterations to their target `Block`. With `BlASC2026-UNIFOLM-WMA-NEWock` restructured as the outer loop, we batched all the corresponding `pox` values per `Block` and dispatched them to the `global_interp` function.

Subsequently, we sorted the `pox` arrays by their spatial offsets. This ensures that adjacent `pox` values are processed within the same batch, thereby effectively exploiting the principle of spatial locality for memory access.

Next, for the `global_interp` function, we directly inlined the `decide3d` helper function—originally responsible for assembling the `ya` variables—into the main body.

Using multiple pox values, we assembled the ya structures in batches and packed them contiguously into the stencil_stage buffer for unified processing.

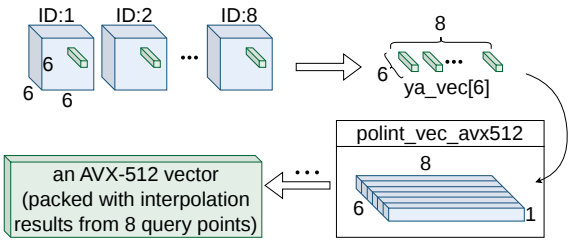


Fig. 23. Interpolation memory diagram

Following this, we modified the original memory layout of ya within stencil_stage to better accommodate the AVX-512 vector instruction set, as illustrated in Fig. 23.

Finally, leveraging AVX-512 vector instructions, we processed multiple ya variables simultaneously and completed the final interpolation within the polint_vec_avx512 function.

To facilitate these optimizations, we modified the function interfaces as follows:

```
static inline void
polint_vec_avx512(
  __m512d  x_vec, const __m512d *ya_vec,
  __m512d *y_out
);
```

Listing 2. Modified polint interface

```
for (int i = 0; i < 6; i++) {
 for (int j = 0; j < 6; j++) {
  for (int k = 0; k < 6; k++) {
    int local_off = i + j*stride_y + k*stride_z;
    __m256i vidx = _mm256_set_epi32(
        7*216 + local_off, 6*216 + local_off,
        5*216 + local_off, 4*216 + local_off,
        3*216 + local_off, 2*216 + local_off,
        1*216 + local_off, 0*216 + local_off
     );
    ya_vec[k] = _mm512_i32gather_pd(
      vidx, arena->stencil_stage, 8
    );
  }
  polint_vec_avx512(z_vec, ya_vec, &res);
 }
}
```

Listing 3. Vectorized assembly of ya and invocation of polint

Full vectorization is achieved in Listing 3 through c_global_interp_batched. This function packs the ya data into SIMD vector ya_vec structures, passing them to polint_vec_avx512 for efficient batched execution.

By combining these acceleration techniques, the cache miss rate of global_interp plummeted from 27% to 3%, effectively neutralizing the memory access bottleneck.

c) *Use Kernel Fusion to Optimize* compute_rhs_bssn*:* In the baseline implementation, the primary overhead of compute_rhs_bssn is memory traffic, not raw compute. The calculations for β derivatives, Γ right-hand side terms, χ and metric RHS, connection corrections, covariant Laplacian corrections, and $A_{\{ij\}}$ contractions are split across multiple independent memory sweeps.

Consequently, the same grid data is redundantly read from and written back to main memory across consecutive phases.For 3D grid computations, these repeated global sweeps constantly flush the cache working set, destroying temporal locality and severely bottlenecking main memory bandwidth.

From a memory access model perspective, the baseline traffic can be formulated as $Q_{\text{base}} = 8N\sum_{s=1}^{S}(R_s + W_s)$, where N is the number of grid points, and R_s, W_s are the read/write volumes of the s-th sweep. Due to inter-phase dependencies, temporary variables like $\mathcal{T}_1, \mathcal{T}_2$ undergo redundant write-read-write cycles, which is the root cause of the function's poor performance.

Algorithm 3: Baseline compute_rhs_bssn (Multi-Pass)

1 **Input:** u_p, u_{stencil}
2 **Pass 1:** Compute $\partial_j \beta^i$ and write temporary tensor cal(T)_1
3 **Pass 2:** Reload cal(T)_1, compute D_β, and update Γ^i_{rhs}
4 **Pass 3:** Update χ_{rhs} and $\gamma_{ij,\text{ rhs}}$ in a separate sweep
5 **Pass 4:** Update $\tilde{\Gamma}^i$ and covariant Laplace terms in a separate sweep
6 **Pass 5:** Compute A^2 and write temporary tensor cal(T)_2
7 **Pass 6:** Reload cal(T)_2, update $A_{ij,\text{ rhs}}, K_{\text{rhs}}, \text{Lap}_{\text{rhs}}$
8 **Output:** $\partial_t u$ (multi-pass write-back)

The core concept of kernel fusion is to collapse the number of memory sweeps and shorten the lifecycle of intermediate variables without altering the underlying discrete mathematical formulations. First, we fused the computation of the β derivatives and the shift-related updates of Γ into a single data path. While the local stencil data is still resident in L1 cache or registers, we synchronously compute $D_\beta = \partial_x \beta^x + \partial_y \beta^y + \partial_z \beta^z$, and immediately update χ_{rhs} and $\gamma_{ij,\text{ rhs}}$. The corresponding χ term maintains its original form $R_\chi = \frac{2}{3}(1\chi)\big((1+\alpha)K - D_\beta\big)$; hence, this is strictly an execution reordering, not an equation rewrite.

Next, we fused the connection updates, covariant Laplacian corrections, and the A^2 contractions. We strived to complete the assembly of $A_{ij,\text{ rhs}}, K_{\text{rhs}}$, and Lap_{rhs} within the same sweep. The critical expressions remain untouched:

$$A^2 = A_{ij}A^{ij} \tag{1}$$

$$R_K = -D^i D_i \alpha + \alpha\left(\frac{1}{3}K^2 + A^2 + 4\pi(\rho + S)\right) \tag{2}$$

$$R_\alpha = -2\alpha K \tag{3}$$

Post-fusion, the memory traffic is approximated by $Q_{\text{fused}} = 8N\big(R_f + W_f\big)$, easily satisfying $Q_{\text{fused}} ll Q_{\text{base}}$. This drasti-

cally boosts the effective arithmetic intensity $I = \frac{F}{Q}$. For implementation, we adopted a Structure-of-Arrays (SoA) layout alongside a vectorized execution path, keeping the scalar fallback numerically identical to guarantee reproducibility.

Algorithm 4: Optimized `compute_rhs_bssn` (Fused)

Input: u_p, u_{stencil}
for each interior grid point p **in parallel do**// Single sweep
  Load local stencil once and keep active data in registers/L1
  Compute $\partial_j \beta^i$ and D_β
  Accumulate shift-related increments to Γ^i_{rhs}
  Update χ_{rhs} and $\gamma_{ij,\text{ rhs}}$
  Update connection and covariant Laplace terms
  $A^2 \leftarrow A_{ij}A^{ij}$ // Fused contraction
  Assemble $A_{ij,\text{ rhs}}, K_{\text{rhs}}, \text{Lap}_{\text{rhs}}$
  Stream $\partial_t u$ back to memory // Write-once
end for
Output: $\partial_t u$ (fused write-back)

In conclusion, kernel fusion transformed an execution model characterized by multiple global traversals and redundant memory spills into an optimized data pipeline relying on localized data residency and delayed write-backs, dramatically reducing memory overhead without sacrificing numerical precision.

d) *Sliding Window to Accelerate `prolong3`:*

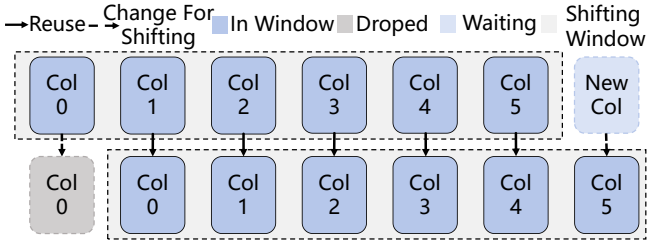


Fig. 24. Sliding window visualization

For the `prolong3` function, the primary logic can be abstracted as shown in Listing 4:

```
for (k ...) {
  for (j ...) {
    for (i ...) {
      //generate vast tmp2 and tmp1
      tmp2 = convolve_z(func(i,j))
      tmp1 = convolve_y(tmp2)}
      //generate result
      funf = convolve_x(tmp1)
    }
  }
}
```

Listing 4. Original 3D convolution logic

From this structure, we identified that the K iterations are independent, yet each K step redundantly recalculates identical intermediate `tmp2` and `tmp1` data, resulting in unnecessary compute overhead.

To mitigate this, as depicted in Fig. 24, we implemented a sliding window approach which shifting `prolong3` from point-by-point evaluation to sliding line-by-line evaluation:

- The computation continuously advances along the X-axis.
- We maintain a local 6×6 window array `tmp2`. For every step forward, we only compute the newly arriving column, reusing the data from the other 5 columns.
- Intermediate variables are shared securely within the same step.

After profiling the optimized `prolong3` function, we observed that the primary bottleneck correctly shifted to the computation of the newly added column during the window advancement, taking about 29.48% of the time.

This aligns perfectly with our sliding window design philosophy: the computational workload is now strictly concentrated on incremental updates, eliminating the vast majority of redundant `tmp1` and `tmp2` derivations.

e) *Use Static Memory Allocation:*

Furthermore, by analyzing the flamegraph in Fig. 19, we noticed severe overhead from repeated dynamic memory allocation and deallocation within the `global_interp` function, represented by the Slate Blue sections in the graph.

To mitigate the overhead incurred by repeated dynamic memory allocation and deallocation, we designed a statically allocated `ThreadArea` to centrally manage all temporary variables.

Consequently, the intermediate buffers required by the `global_interp` function are allocated only once throughout the entire program lifecycle.

Ultimately, after the optimization was completed, we could no longer see any time spent on memory allocation and deallocation on the new flame graph.

f) *Others:*

`TwoPunctureABE` Optimization:

Compared with ABE, TwoPunctureABE accounts for a smaller fraction of total runtime; however, it lies on the critical path of initial-data construction in numerical relativity and therefore directly affects the startup efficiency of subsequent evolution as well as the throughput of parameter sweeps. Our profiling indicates that the dominant issue in this stage is not an unusually expensive single-point kernel. Rather, the iterative solver repeatedly performs structurally similar grid sweeps, temporary data assembly, and inter-stage waiting, which together lead to low computational intensity, high memory-access overhead, and underutilization of parallel resources. This bottleneck is characteristic of workflow-level

inefficiency; accordingly, optimization should target the data path and execution pipeline, rather than only local arithmetic tuning.

Based on this diagnosis, our overall strategy is to shorten the critical path and increase useful work per memory access, while preserving the discretization scheme and governing equations.

- Parallel expansion: apply stable multi-core parallelization to sweep phases with exploitable concurrency, thereby reducing serial waiting and inter-stage idle time.
- Data-path optimization: shorten intermediate-data lifetimes and improve data reuse by reducing redundant construction/reads, improving cache locality, and introducing lightweight specialization for high-frequency generic kernels to lower control-flow and indexing overhead.

In essence, these strategies transform a multi-stage execution pattern with repeated read/write cycles into one with fewer passes and higher reuse, which improves portability across similar workloads.

Empirically, this stage achieves an approximately **3.18x** speedup and exhibits better scalability under multi-core configurations. More importantly, these optimizations complement those for the main ABE pipeline: the former reduces initialization and preprocessing tail costs, while the latter reduces per-step cost in the core evolution loop, jointly improving end-to-end workflow efficiency.

GPU Version:

During testing, we discovered that the original GPU implementation exhibited severe precision discrepancies compared to the CPU baseline, far exceeding the 1% accuracy tolerance limit. Furthermore, the legacy GPU code suffered from excessively fine-grained kernel invocations, which led to frequent Host-to-Device and Device-to-Host memory transfers, high kernel launch overheads, and severe synchronization penalties.

Consequently, we rectified the precision bugs in the original GPU version and comprehensively rewrote its hotspot operators, applying aggressive kernel-level fusion and optimizations. Because the GPU optimization philosophy closely mirrors that of the CPU version, we omit exhaustive details here. The overarching operator optimization strategies are summarized as follows:

1) In the `compute_rhs_bssn` function:
   The original implementation dispatched fields separately, executing the `lopsided` and `kodiss` functions independently, which led to high data transfer and synchronization overheads. To resolve this, we fused `lopsided` and `kodiss` into a single kernel, prioritizing the fused execution path for overlapping fields.
2) In the `global_interp` function:
   The legacy code performed point-by-point interpolation, point-by-point data transfers, and individual kernel launches, resulting in suboptimal throughput. Therefore, we refactored this logic to support batched interpolation: we first construct the stencils in bulk, and subsequently invoke a batched stencil kernel to process multiple points concurrently.
3) In the `prolong3` function:
   The original implementation exhibited poor throughput under boundary conditions and fine-grained invocations. To address this, we developed a specialized kernel tailored for the `Cell+ghost_width==3` scenario. Using the fine-grid output points as the parallelization granularity, we executed the $6 \times 6 \times 6$ stencil interpolation convolutions directly on the GPU. Additionally, we performed symmetric boundary expansion prior to the kernel execution, thereby mitigating the overhead of point-by-point calls and intermediate data movement.

Ultimately, building upon the corrected GPU implementation, we achieved a speedup of **8.3x**. Specifically, when running with 64 processes, the execution time was reduced from a baseline of approximately **151.03s** to an optimized time of roughly **18.21s**.

F. Result & Analysis

In addressing the overall problem, we conducted an in-depth analysis of both the original CPU and GPU implementations of the AMSS-NCKU code. As discussed in the preceding sections, both versions exhibit meaningful optimization potential, and we therefore performed moderate, implementation-specific optimizations for each.

When selecting the final baseline and optimized implementations for quantitative reporting, we considered two major issues observed during reproduction of the original GPU version: (1) its total runtime on our platform was significantly longer than that of the reproduced CPU version, and (2) it did not achieve numerical accuracy consistent with the CPU implementation. For these reasons, the original GPU version is unsuitable as the primary baseline for presenting final optimization gains, although it remains appropriate for fair within-GPU comparisons.

Although our optimized GPU implementation delivers substantial speedups, establishing a baseline that is both effective and methodologically fair is difficult in this setting. A fallback option would be to compare the optimized GPU version directly against the CPU baseline; however, such a comparison would confound algorithmic effects with hardware heterogeneity and therefore lacks fairness. We thus do not adopt this approach.

Accordingly, we use the original CPU implementation as the baseline and compare it objectively against our optimized CPU implementation. For completeness, we also report the optimized GPU results in earlier sections as supplementary

reference. The following provides a concise summary of the CPU-side optimizations and final outcomes.

Synthesizing all the acceleration methodologies detailed above, the ablation study mapping the cumulative speedup for a single `ABE Timestep` is shown below:

TABLE XIV
ABLATION STUDY OF OPTIMIZATION TECHNIQUES

Optimization Stage	Total Time (sec)	SpeedUp
Baseline	13.5544	1.00x
Batch + SIMD	6.98680	1.94x
Kernel Fusion	11.2020	1.21x
MPI + OpenMP	10.0403	1.35x
Fusion + Batch + SIMD	5.29469	2.56x
Sliding window	10.5894	1.28x
Total End-to-End	3.12834	**4.33x**

Additionally, the correctness of the optimized program is subject to the following two requirements:

Accuracy Requirement

- The Root Mean Square (RMS) error of the black hole trajectories between the optimized and pre-optimized versions must remain below 1% for each independent coordinate. Note that the Z-axis is excluded from this evaluation.

$$\sqrt{\frac{1}{M}\sum_{i=1}^{M}\left(\frac{|c_{1i}-c_{2i}|}{\max(|c_{1i}|,|c_{2i}|)}\right)^2}$$

  where c_{1i} and c_{2i} represent the specific black hole coordinate values (x or y) for BH1 and BH2 before and after optimization, respectively. The solution is considered valid only if the RMS error for all four dimensions ($\mathrm{BH1}_x, \mathrm{BH1}_y, \mathrm{BH2}_x, \mathrm{BH2}_y$) is less than 1%.
- Constraint Violation: The absolute value of the ADM constraint violation on the outermost grid level must not exceed 2. This metric serves as a quantitative standard to measure the overall accuracy of the simulation.

While the Constraint Violation can be directly observed from Fig. 28, we believe that visual comparisons between Fig. 27 and Fig. 30 are insufficient for verifying the Accuracy Requirement as they lack intuitive precision.

Consequently, we have implemented a verification program that compares the bssn_BH.dat output files from both the baseline and optimized versions. The core logic of the verification script is as follows:

```
M = min(len(data1), len(data2))
# Cols: 1,2 for BH1(x,y); 4,5 for BH2(x,y).
# Z-axis (3,6) excluded.
indices = [1, 2, 4, 5]

def get_rms(ref, test):
    diff = np.abs(ref - test)
    denom = np.maximum(np.maximum(np.abs(ref),
np.abs(test)), 1e-14)
    return np.sqrt(np.mean((diff / denom)**2))

# Calculate RMS for each coordinate;
# all must be < 1.0%
errors = [get_rms(data1[:M, i], data2[:M, i]) * 100
for i in indices]
passed = all(err < 1.0 for err in errors)
```

Listing 5. Core implementation of RMS-based trajectory accuracy check

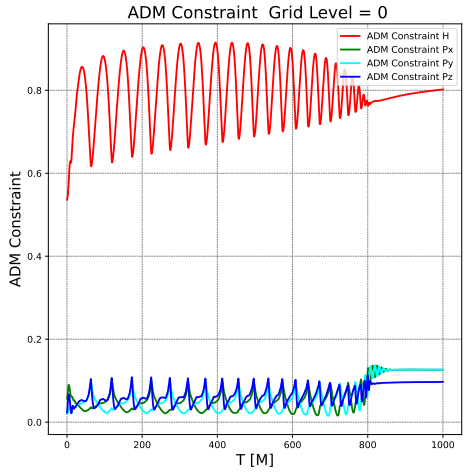


Fig. 25. Baseline ADM Constraint

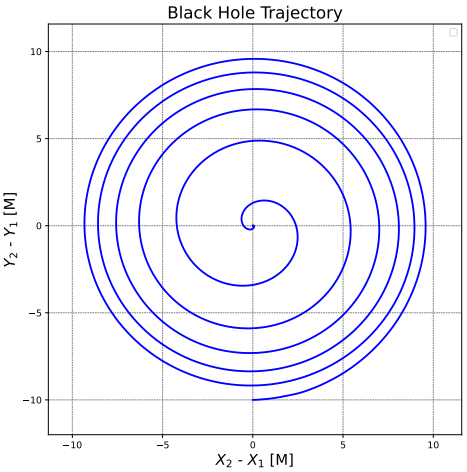


Fig. 26. Baseline XY Trajectory

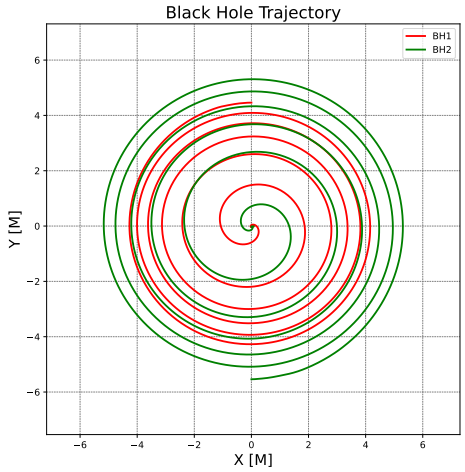


Fig. 27. Baseline Trajectory

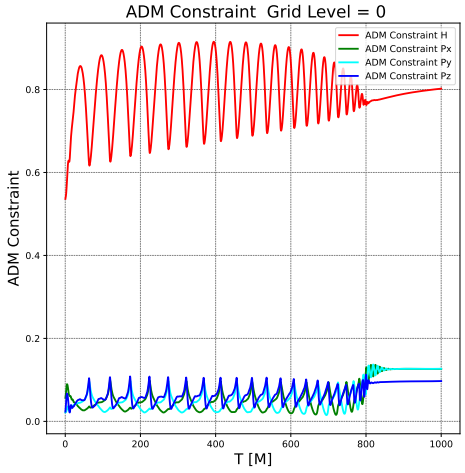


Fig. 28. Optimized ADM Constraint

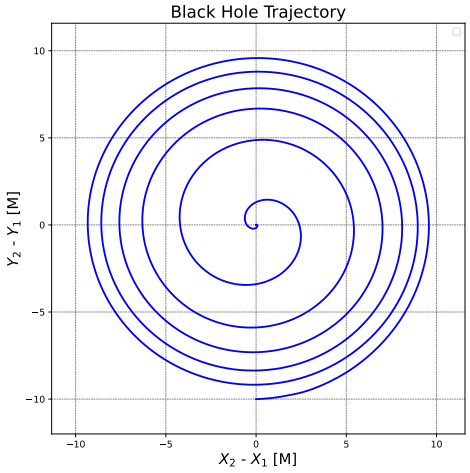


Fig. 29. Optimized XY Trajectory

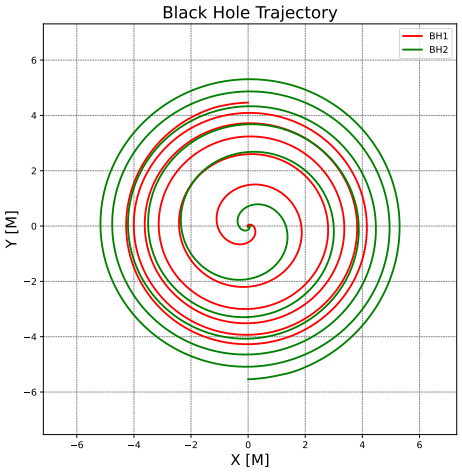


Fig. 30. Optimized Trajectory

G. Conclusion

Ultimately, we successfully completed the optimization of the AMSS-NKCU project. The CPU baseline execution time was **33,183.5s**, which was reduced to **7800.6s** post-optimization, achieving a speedup ratio of **4.25x**. Furthermore, the error margin between the optimized results and the original project results is maintained at 0.6%, fully satisfying the requirements of the challenge.

References

[1] "HPL - A Portable Implementation of the High-Performance Linpack Benchmark for Distributed-Memory Computers." Accessed: Jan. 17, 2025. [Online]. Available: https://www.netlib.org/benchmark/hpl/

[2] "HPCG Benchmark: A New Metric for Ranking High Performance Computing Systems." Accessed: Feb. 28, 2026. [Online]. Available: https://www.hpcg-benchmark.org/

[3] Unitree, "UnifoLM-WMA-0: A World-Model-Action (WMA) Framework under UnifoLM Family." 2025.

[4] Y. Huang *et al.*, "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism." [Online]. Available: https://arxiv.org/abs/1811.06965

[5] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[6] B. Lefaudeux *et al.*, "xFormers: A modular and hackable Transformer modelling library." 2022.

[7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[8] X. Ma, G. Fang, and X. Wang, "DeepCache: Accelerating Diffusion Models for Free," in *The IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[9] "Perf: Linux Profiling with Performance Counters ." Accessed: Dec. 01, 2024. [Online]. Available: https://perfwiki.github.io/main/